TECHNICAL SPECIFICATION & REFERENCE MANUAL V3.0.0

ZIMDRIMS Backend API & Technical Reference (Basic Introductory Sets)

Anticipatory Action & Disaster Risk Management Core Services (Including Input/Output Specs)

Architectural Overview & Design Pattern

Asynchronous Node.js & Express runtime interfacing with Microsoft SQL Server.

Node.js & Nginx

Express backend running on port 3004 behind Nginx proxy with enforced TLS.

MSSQL Connection

Relational DB pooling for zimdrims_db & LogicAdminDB with 60s timeout.

Unified RPC Pattern

Replaces standard REST endpoints with robust POST routing for database procedures.

Production Routing & Base URL

Standardized environment setup for client integration.

Network Parameters

- **Production URL:** https://dcp.dikarabo.co.za
- **Internal Host:** 41.76.212.195 (Port 3004)
- **Security:** Nginx Reverse Proxy with TLS/SSL

Core Routing Engines

POST /getJSONData
Operational reads, filters, and standard queries.

POST /getAdminJSONData
Elevated admin actions and database writes.

Endpoint Deep Dive: POST /getJSONData

Handling operational queries and SQL JSON chunk reassembly.

Execution Pipeline

1. Express parses body (schema, spName, payload).
2. Acquires connection pool to zimdrims_db.
3. Binds payload object to SQL parameter @jsonInput (NVARCHAR(MAX)).
4. Executes `EXEC [schema].[spName] @jsonInput`
5. Reassembles ~2033 character SQL chunk rows into valid JSON string.

```
// Payload Structure Example
{
 "schema": "dbo",
 "spName": "select_districts",
 "payload": [
  { "province_id": 2 }
 ]
}
```

Endpoint Deep Dive: POST /getAdminJSONData

Reserved for administrative actions, user provisioning, and secure writes.

Administrative Execution

1. Ingests admin management payloads.
2. Routes toward write stored procedures (e.g.,).
   `sp_upsert_command_user`
3. Executes with @jsonInput binding.
4. Returns direct recordset confirmation statuses or newly generated identity keys.

```
// Admin Upsert Payload Example

{
  "schema": "dbo",
  "spName": "sp_upsert_command_user",
  "payload": [
    { "user_id": 14, "forename": "Tinashe", "surname": "Moyo",
"systemRole": 2, "is_enabled": true }
  ]
}
```

Stored Procedures: Read Operations & Expected Outputs

Modular read procedures with input parameters and expected JSON output structure.

[dbo].[select_provinces]

Input: {} (None)

Output: Array of provinces with aggregated ward population (id, name, code, male, female, total).

[dbo].[select_districts]

Input: province_id (INT), provinceIds (Array)

Output: Array of districts with demographic summaries (id, name, code, province_name, male, female, total).

[dbo].[select_hazards]

Input: {} (None)

Output: Array of active hazard modules (id, code, name, icon_name, color, descr).

[dbo].[select_hazard_categories]

Input: {} (None)

Output: Array of disaster risk classification categories.

Stored Procedures: Authentication & User Lookups

User verification parameters and nested JSON access privilege outputs.

[dbo].[select_login]

Input: email (STRING), password (STRING)

Output: Nested JSON user profile with stageAccess, deptAccess, hazardAccess, provinceAccess, districtAccess, and role.

[dbo].[sp_get_command_users]

Input: user_id (INT, Opt), role_id (INT, Opt)

Output: Array of command users with nested privilege arrays encoded as JSON.

[dbo].[select_user_exists]

Input: email (STRING)

Output: Status record (user_exists: 1/0, has_password: 1/0).

[dbo].[select_all_hazard_items]

Input: hazard_id (INT, Optional)

Output: Active hazard mappings joined with taxonomy lookups and modules.

Stored Procedures: Write & Admin Operations

Administrative provisioning parameters and execution status outputs.

[dbo].[sp_upsert_command_user]

Input: user_id, forename, surname, email, mobile, systemRole, depts, stageAccess, hazardAccess, provinceAccess, districtAccess

Output: confirmation object ({ user_id, status: 'Success' }).

[dbo].[sp_upsert_aa_contact]

Input: id, forename, surname, role_id, email, mobile, department_id, province_id, district_id, hazard_id

Output: confirmation record ({ user_id, status: 'success' }).

[dbo].[reset_password]

Input: user_id (INT)

Output: confirmation status ({ done: 1 }).

[dbo].[sp_upsert_hazard_item]

Input: mapping_id, hazard_id, item_name_id, is_active

Output: confirmation object ({ mapping_id, hazard_id, item_name_id, status: 'Success' }).

Production API Client Library (`apiClient.js`)

Asynchronous JavaScript wrapper interfacing with https://dcp.dikarabo.co.za

```
const API_BASE_URL = 'https://dcp.dikarabo.co.za';
export async function getJSONData(schema, spName, payload = {})
 {
        const response = await fetch(`${API_BASE_URL}/getJSONData`,
        {       method: 'POST', headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ schema, spName, payload: [payload] })
});
if (!response.ok)
throw new Error(`HTTP error! status: ${response.status}`); return await response.json();
 }

export const fetchProvinces = () => getJSONData('dbo', 'select_provinces');

export const authenticateUser = (email, password) => getJSONData('dbo', 'select_login', { email, password });
```


```
const API_BASE_URL = 'https://dcp.dikarabo.co.za';
export async function getJSONData(schema, spName, payload = {})
{
        const response = await fetch(`${API_BASE_URL}/getJSONData`,
                                        { method: 'POST', headers: { 'Content-Type': 'application/json' },
                                          body: JSON.stringify({ schema, spName, payload: [payload] })
});
 if (!response.ok) throw new Error(`HTTP error! status: ${response.status}`);
return await response.json(); }


export const fetchProvinces = () => getJSONData('dbo', 'select_provinces');


export const authenticateUser = (email, password) => getJSONData('dbo', 'select_login', { email, password });
```